Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Program: Software Engineering

Operating Systems

Laboratory Work No. 6

Inside the xv6 File System

Track B: Build Your Own Small OS (xv6)

Elaborated by:
st. gr. FAF-241
Ermicev Sofia

Verified by:
Patricia Reițman

Chișinău – 2026

Contents

1 Introduction **2**

2 Part 1. Reading What xv6 Shows **2**
2.1 Observations 3

3 Part 2. The `mystat` Program **3**
3.1 Observations 5

4 Part 3. Measuring File-System Limits **5**
4.1 Observations 6

5 Part 4. The Recursive `find` Program **6**
5.1 Observations 9

6 Conclusion **9**

1. Introduction

The purpose of this laboratory work is to examine the internal organization of the xv6 file system and connect the concepts of directories, inodes, hard links, block addressing, and file-system limits with the actual xv6 source code. The work also includes two user programs: `mystat`, which displays inode metadata, and `find`, which recursively traverses the directory tree.

All experiments were performed in the existing `/x̃v6-riscv` tree. New user programs were added to the `user/` directory and registered in the `UPROGS` list in the Makefile.

2. Part 1. Reading What xv6 Shows

The file types were first decoded from `kernel/stat.h`:

```
$ grep -n "T_DIR\|T_FILE\|T_DEVICE" kernel/stat.h
1:#define T_DIR    1 // Directory
2:#define T_FILE   2 // File
3:#define T_DEVICE 3 // Device
```

The root directory was listed in xv6. The beginning of the output was:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
echo           2 4 34312
...
console        3 25 0
```

Each line printed by xv6 `ls` contains the name, type, inode number, and size in bytes.

A new directory `a` was then inspected:

```
$ ls a
.              1 26 32
..             1 1 1024
```

The directory-entry structure and the maximum stored name length were inspected in `kernel/fs.h`:

```
struct dirent {
  ushort inum;
```

```
  char name[DIRSIZ] __attribute__((nonstring));
};

#define DIRSIZ 14
```

2.1. Observations

1. Type 1 means directory (`T_DIR`), type 2 means regular file (`T_FILE`), and type 3 means device (`T_DEVICE`).
2. The root directory has inode number **1**. In the root directory, both `.` and `..` have inode 1 because `.` refers to the current directory and the root has no parent above itself, so its `..` also points back to root.
3. An empty directory `a` has size 32 bytes and contains exactly two entries: `.` and `...` A `struct dirent` contains a 2-byte `ushort inum` and a 14-byte name, so one directory entry occupies **16 bytes**. Two entries therefore occupy **32 bytes**.

3. Part 2. The `mystat` Program

The program `user/mystat.c` was implemented using the xv6 `stat()` call and `struct stat`. It converts numeric types to readable names and prints the inode number, link count, and size.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char *
typename(int t)
{
    if(t == T_DIR)
        return "directory";
    if(t == T_FILE)
        return "file";
    if(t == T_DEVICE)
        return "device";
    return "?";
}

int
```

```
main(int argc, char *argv[])
{
    struct stat st;

    for(int i = 1; i < argc; i++){
        if(stat(argv[i], &st) < 0){
            fprintf(2, "mystat: cannot stat %s\n", argv[i]);
            continue;
        }

        printf("%s: %s, inode %d, links %d, size %ld bytes\n",
               argv[i], typename(st.type), st.ino, st.nlink,
               (long)st.size);
    }

    exit(0);
}
```

The program was added to the Makefile and successfully built. It was first tested on a file, a directory, and a device:

```
$ mystat README . console
README: file, inode 2, links 1, size 2441 bytes
.: directory, inode 1, links 1, size 1024 bytes
console: device, inode 26, links 1, size 0 bytes
```

A hard link was then created:

```
$ ln README r2
$ mystat README r2
README: file, inode 2, links 2, size 2441 bytes
r2: file, inode 2, links 2, size 2441 bytes
```

After removing the original name, the second name continued to access the same inode:

```
$ rm README
$ mystat r2
r2: file, inode 2, links 1, size 2441 bytes
```

The unlink implementation was inspected in `kernel/sysfile.c`. The relevant part is:

```
if (ip->type == T_DIR) {
    dp->nlink--;
```

```
    iupdate(dp);
}
...
ip->nlink--;
iupdate(ip);
iunlockput(ip);
```

The on-disk inode structure was inspected in `kernel/fs.h`:

```
struct dinode {
  short type;
  short major;
  short minor;
  short nlink;
  uint size;
  uint addrs[NDIRECT + 1];
};
```

3.1. Observations

1. After `ln README r2`, both names had the same inode number **2**, the same size **2441 bytes**, and their link count became **2**. After `rm README`, `r2` still referred to inode 2 with the same size, while the link count decreased to **1**. Thus, the name changed, but the inode and data did not.
2. In `sys_unlink`, the link count of the file inode is decremented by `ip->nlink--`. The file's blocks are not freed merely because one name is removed; they can be reclaimed only when the inode no longer has directory links and its final in-memory reference is released.
3. The xv6 on-disk inode contains the fields `type`, `major`, `minor`, `nlink`, `size`, and `addrs[NDIRECT + 1]`. Unlike the Linux experiment from Lab 5, the xv6 inode used here does not contain Unix permission bits, and this xv6 version has no symbolic-link type in `kernel/stat.h`.

4. Part 3. Measuring File-System Limits

The constants controlling the basic file-system limits were read from `kernel/fs.h`:

```
$ grep -n "define BSIZE\|define NDIRECT\|define NINDIRECT\|define
   DIRSIZ" kernel/fs.h
5:#define BSIZE   1024 // block size
26:#define NDIRECT   12
```

```
27:#define NINDIRECT (BSIZE / sizeof(uint))
54:#define DIRSIZ 14
```

A clean file-system image produced the following layout information:

```
$ make clean
$ make fs.img 2>&1 | grep nmeta
nmeta 47 (boot, super, log blocks 31, inode blocks 13,
bitmap blocks 1) blocks 1953 total 2000
```

The name-length limit was measured in xv6:

```
$ mkdir averyveryverylongname
$ ls
...
averyveryveryl 1 28 32
```

4.1. Observations

1. The requested name `averyveryverylongname` appears as `averyveryveryl`. The stored name is limited to **14 characters** by `DIRSIZ = 14`.
2. A file inode has 12 direct block addresses and one indirect block. With `BSIZE = 1024` bytes and a 4-byte `uint`, the indirect block can contain

$$\text{NINDIRECT} = \frac{1024}{4} = 256$$

block addresses. Therefore the largest file is

$$(12 + 256) \times 1024 = 274432 \text{ bytes} = 268 \text{ KiB}.$$

3. The complete file-system image contains **2000 blocks**. The build reports **47 metadata blocks**: 1 boot block, 1 superblock, 31 log blocks, 13 inode blocks, and 1 bitmap block. The remaining **1953 blocks** are data blocks.

5. Part 4. The Recursive `find` Program

The second user program, `user/find.c`, recursively walks a directory tree. It opens the requested directory, reads one `struct dirent` at a time, skips unused entries and `./..`, builds each full path, compares the last component with the requested name, and recursively descends into subdirectories.

```
#include "kernel/types.h"
```

```c
#include "kernel/stat.h"
#include "kernel/fs.h"
#include "user/user.h"
#include "kernel/fcntl.h"

void
find(char *path, char *target)
{
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;

    if((fd = open(path, O_RDONLY)) < 0){
        fprintf(2, "find: cannot open %s\n", path);
        return;
    }

    if(fstat(fd, &st) < 0){
        fprintf(2, "find: cannot stat %s\n", path);
        close(fd);
        return;
    }

    if(st.type != T_DIR){
        close(fd);
        return;
    }

    if(strlen(path) + 1 + DIRSIZ + 1 > sizeof(buf)){
        fprintf(2, "find: path too long\n");
        close(fd);
        return;
    }

    strcpy(buf, path);
    p = buf + strlen(buf);
    if(p > buf && p[-1] != '/')
        *p++ = '/';

    while(read(fd, &de, sizeof(de)) == sizeof(de)){
        if(de.inum == 0)
            continue;
```

```
        char name[DIRSIZ + 1];
        memmove(name, de.name, DIRSIZ);
        name[DIRSIZ] = 0;

        if(strcmp(name, ".") == 0 || strcmp(name, "..") == 0)
            continue;

        memmove(p, de.name, DIRSIZ);
        p[DIRSIZ] = 0;

        if(stat(buf, &st) < 0)
            continue;

        if(strcmp(name, target) == 0)
            printf("%s\n", buf);

        if(st.type == T_DIR)
            find(buf, target);
    }

    close(fd);
}
```

The Makefile was updated with both new programs:

```
$U/_mystat\
$U/_find\
```

A clean build successfully compiled `user/find.c` and produced `user/_find`. The required recursive test was then performed:

```
$ mkdir a
$ mkdir a/b
$ echo x > a/b/r2
$ echo y > a/r2
$ ln cat r2
$ find . r2
./a/b/r2
./a/r2
./r2
```

A second test searched for the directory named `b` below `a`:

```
$ find a b
```

```
a/b
```

5.1. Observations

The output confirms that `find` traverses nested directories recursively and prints every matching path. The program does not recurse into `.` or `..`, preventing an infinite cycle. It also skips entries whose inode number is zero and closes each directory descriptor after use, which is important because xv6 provides only a small number of open file descriptors per process.

6. Conclusion

This laboratory work demonstrated how the xv6 file system implements familiar Unix concepts using a small and readable code base. Directory listings exposed the direct relationship between names, inode numbers, file types, and sizes. The `mystat` experiment showed that hard links are multiple directory names for the same inode and that unlinking one name only decreases the inode's link count.

The file-system constants revealed xv6's deliberate limits: 14-character directory names, 1024-byte blocks, 12 direct blocks, and one indirect block, giving a maximum file size of 268 KiB. The disk-image layout also showed the separation between metadata and data blocks. Finally, the recursive `find` program combined directory entries, inode metadata, system calls, path construction, and recursion into a practical file-system utility. Overall, the work connected xv6 source code directly to observable file-system behavior.